OOP DESIGN PRINCIPLES

Single Responsibility Principle (SRP)

A class should have **only one reason to change**.

Smell: one class does storage + logic + formatting.
Fix: extract a class per responsibility.
Lab: Program2.1 (bad) → Program2.5 (good)

Open-Closed Principle (OCP)

Open for extension, closed for modification.

Smell: adding a new type requires editing existing if/switch.
Fix: define abstract interface; add new subclass.
Lab: Program4.1 (bad) → Program4.4 (interface)

Law of Demeter (Least Knowledge)

Only talk to **immediate friends** — not strangers.

*Smell: **a.getB().getC().doThing()** — chain of dots.*
*Fix: **a.doThing()** — delegate.*
Returning raw pointer = exposing internal state.
Fix: return a copy (`new Date(*birthdate)`).
Lab: Program5.7, 5.8, 5.9

RECURSION

Base case = smallest input, solved directly. **Recursive case** = shrink + recurse + combine.

```
int largest(vector<int> v) {
  if (v.size()==1) return v[0];   // base
  int first = v[0];
  v.erase(v.begin());
  return max(first, largest(v));  // recurse
}
```

Towers of Hanoi: *Move n-1 out of way → move big disk → move n-1 back. Cost = 2^n-1 moves.*
Quicksort: *Partition around pivot → sort left → sort right. Base: size < 2.*

BACKTRACKING

Recursion + **undo step**. Try a choice → recurse → undo if it fails.

```
// Universal template
for each candidate:
  if valid(candidate):
    apply(candidate)       // PLACE
    recurse()              // GO DEEPER
    undo(candidate)        // BACKTRACK
```

N-Queens: *Place queen → recurse to next col → `occupied[r] [c]=false` on return.*
Sudoku: *Try digits 1–9 → recurse → `grid[r][c]=0` if all fail → return false.*

MULTITHREADING

Race condition: *Two threads modify shared data with no sync → garbled result.*
thread t(fn, arg); *then **t.join()** — always join or program crashes.*

mutex + lock_guard (RAII) ← preferred

```
mutex m;
void fn() {
  lock_guard<mutex> g(m); // auto-unlock on scope exit
  /* critical section */
}
```

Manual lock/unlock: *risky — exception skips unlock → deadlock.*
yield(): *hint to OS to run another thread. NOT a lock. Does not release mutex.*

Reader-Writer: shared_mutex

```
shared_mutex m;
// Writer (exclusive):
unique_lock<shared_mutex> wl(m);
// Reader (shared — concurrent OK):
shared_lock<shared_mutex> rl(m);
```

Deadlock — 4 conditions (break any 1 to prevent)

- Mutual exclusion
- Hold and wait
- No preemption
- Circular wait ← break this: always acquire locks in fixed order

Producer-Consumer: *mutex-protected queue; check is_full before push, is_empty before pop.*
atomic<T>: *thread-safe read/write without a mutex (used in DiningPhilosophers).*

BEHAVIORAL PATTERNS

Template Method `Behavioral` **· Inheritance**

Base class defines **fixed algorithm sequence**. Pure virtual steps = subclasses fill in.

Concrete steps (header/footer) same for all. Abstract steps (acquire/analyze/print) vary.
vs Strategy: Template Method = inheritance, can't swap at runtime.
Strategy = composition, can.

```
class GameReport {
  void generate() {     // fixed skeleton
    print_header();     // concrete
    acquire_data();     // pure virtual
    analyze_data();     // pure virtual
    print_report();     // pure virtual
    print_footer();     // concrete
  }
};
```

Lab: Program8.2

Strategy `Behavioral` **· Composition**

Context holds a **pointer to strategy interface**. Algorithm swappable at runtime.

Add new algorithm = new strategy class. No change to context.

```
class Sport {
  PlayerStrategy* ps;
  VenueStrategy*  vs;
  void set_player_strategy(PlayerStrategy* s)
    { ps = s; } // runtime swap
  string recruit() { return ps->strategy(); }
};
```

Lab: Program8.4

Observer `Behavioral`

Subject notifies all registered Observers when state changes. 1-to-many. Observers are decoupled.

attach / detach / notify *in Subject.* ***update()*** *in Observer.*
Push: *subject sends data in update() args.* ***Pull:*** *observer queries subject after notification.*

```
class Subject {
  vector<Observer*> obs;
  void notify(...) {
    for(auto* o:obs) o->update(...);
  }
};
```

Lab: Program12.2

State `Behavioral`

Object delegates all behavior to current **State object**. State transitions return the next state.

Replaces giant switch(state) in every method. Each state class is self-contained.

```
class TicketMachine {
  State* current = new READY();
  void insert_card() {
    current = current->insert_card();
  } // current state decides what happens
};
class READY : public State {
  State* insert_card() override {
    return new VALIDATING(); // transition
  }
};
```

Lab: Program13.2

Iterator `Behavioral`

Uniform **has_next() / next()** interface over any collection type (array, vector, map).

Client traversal code never changes regardless of collection type.
Lab: Program11.2

Visitor `Behavioral`

Add operations to stable class hierarchy without modifying it. New operation = new Visitor class.

Double dispatch: *node.accept(v) → v.visit_NodeType(this).*
Use when: hierarchy stable, operations change often. Don't use when: hierarchy changes often.
Lab: Program11.4

STRUCTURAL PATTERNS

Adapter `Structural`

Converts one interface to another. Wraps an incompatible object.

Inner adapter class inherits expected interface, wraps the foreign object, translates method calls.

```
class ConvertedData : public AttendanceData {
  ConvertedData(FootballInfo* f);
  int get_attendance() override
    { return f->get_fans(); } // translate
};
```

Lab: Program10.2 · vs Decorator: Adapter changes interface, Decorator adds behavior.

Facade `Structural`

Single simplified entry point to a complex subsystem.

Client only knows the Facade. Subsystem classes are hidden.

```
class FundRaiser {            // Facade
  void do_fund_raising() {
    Alumni a;    a.send_solicitations();
    BoosterClubs b; b.schedule_meetings();
    Students s;  s.collect_fees();
  }
};
```

Lab: Program10.5

Composite `Structural`

Tree of leaf + group objects. Client uses **uniform interface** on both.

Group's get_cost() recursively sums children. Client doesn't know if it's a leaf or tree.

```
class ProvisionItem { virtual double get_cost()=0; };
class ProvisionGroup : public ProvisionItem {
  vector<ProvisionItem*> children;
  double get_cost() override {
    double t=0;
    for(auto* c:children) t+=c->get_cost();
    return t; // recursive sum
  }
};
```

Lab: Program14.4

Decorator `Structural`

Adds behavior dynamically by wrapping. **Inherits AND holds** same base type → enables chaining.

vs Subclassing: N features × M base types = N+M classes (not N×M subclasses).

```
class Decorator : public Ticket { // inherits
  Ticket* wrapped;                // holds same type
  double get_cost() override {
    return my_price + wrapped->get_cost();
  }
};
Ticket* t = new VIP(new Party(new BaseTicket()));
```

Lab: Program14.6

CREATIONAL PATTERNS

Factory Method `Creational`

Creator defines virtual **create_X()**. Subclass overrides to instantiate the right product. Caller never names concrete class.

```
// Form 1 — static
static Toy* make(Kind k) { switch(k) { ... } }

// Form 2 — virtual
class AthleticsDept {
  virtual Sport* create_sport(Type) = 0; // factory method
};
class VarsityDept : public AthleticsDept {
  Sport* create_sport(Type t) override { return new VarsitySport(t); }
};
```

Lab: Program7.7, Program9.2

Abstract Factory `Creational`

Creates a **family of related objects**. Swap factory = swap entire family.

vs Factory Method: one product vs whole family. Abstract Factory has multiple make_X() methods.

```
class ProvisionsFactory {
  virtual PlayerStrategy* make_players(Type)=0;
  virtual VenueStrategy*  make_venue()=0;
};
// VarsityFactory → DraftPlayers + Stadium
// IntramuralFactory → OpenTryouts + Gym
```

Lab: Program9.3

Singleton `Creational`

Exactly **one instance**. Private constructor. Static get_instance(). Delete copy/assign.

```
class Logger {
public:
  static Logger& get_instance() {
    static Logger inst; // created once
    return inst;
  }
  Logger(const Logger&) = delete;
  Logger& operator=(const Logger&) = delete;
private:
  Logger() {}
};
```

Lab: Program14.1

HOW TO IDENTIFY THE RIGHT PATTERN

If the problem says…	Use…
"same steps, different implementations"	Template Method
"swap algorithms at runtime"	Strategy
"notify multiple objects on change"	Observer
"object behavior changes per state"	State
"traverse without knowing collection type"	Iterator
"new operation, don't touch classes"	Visitor
"incompatible interfaces, can't modify either"	Adapter
"hide complexity of many subsystems"	Facade
"part-whole tree, uniform interface"	Composite
"add features dynamically, avoid subclass explosion"	Decorator
"centralize object creation, hide class names"	Factory Method
"create families of related objects together"	Abstract Factory
"only one instance ever"	Singleton

EASILY CONFUSED PATTERNS

Template Method vs Strategy

- TM: uses **inheritance**. Skeleton fixed at compile time.
- Strategy: uses **composition**. Swappable at runtime.
- TM: subclass overrides steps. Strategy: inject strategy object.

Factory Method vs Abstract Factory

- FM: **one product**, one virtual create_X().
- AF: **family of products**, multiple factory methods in one factory object.
- AF: swap factory → swap entire product family consistently.

Adapter vs Decorator

- Adapter: **changes interface** to match expected. Wraps incompatible object.
- Decorator: **adds behavior**. Same interface in and out. Chainable.
- Decorator: inherits AND holds same base type.

Composite vs Decorator

- Composite: **tree**, 0-to-many children, aggregation.
- Decorator: **chain**, exactly one wrapped object, adds behavior.

Observer vs State

- Observer: external listeners notified of subject's change.
- State: object's OWN behavior changes based on internal state.

Visitor vs Iterator

- Iterator: **traversal** only — has_next/next.
- Visitor: **operations** on each element — type-specific behavior.

DESIGN PROBLEM APPROACH

Step 1 — Identify the problem type

- Is there duplicated logic across classes? → Template Method or Strategy
- Does adding a new type require editing existing code? → OCP violation → Factory or interface
- Does one class do too many things? → SRP violation
- Is there a chain of method calls? → Demeter violation
- Does object behavior depend on internal state? → State pattern
- Do multiple objects need to react to one object's changes? → Observer
- Are features being added via flags/conditionals? → Decorator
- Is there a complex subsystem the client shouldn't see? → Facade
- Are two incompatible interfaces involved? → Adapter

Step 2 — Draw the class hierarchy

- Abstract base = defines the interface / skeleton
- Concrete classes = the implementations
- Composition (has-a) vs Inheritance (is-a)
- Composition preferred for flexibility (Strategy, Observer, Decorator)

Step 3 — State which principle is satisfied

- Factory + interface → OCP (extend without modify)
- Extracted class → SRP
- Delegate instead of chain → Law of Demeter
- Composition over inheritance → flexible, testable

MODERN C++ QUICK REF

Lambda Expressions

```
// [capture](params) -> return { body }
auto f = [](int x) { return x * 2; };
// Capture local var:
int n = 5;
auto g = [n](int x) { return x + n; };  // by value
auto h = [&n](int x){ n += x; };        // by reference
// With STL:
sort(v.begin(), v.end(), [](int a,int b){ return a<b; });
```

***[]*=nothing *[=]*=all by val *[&]*=all by ref**

Functor — overloads operator()

```
class RandomInt {
  int min, max;
public:
  RandomInt(int a, int b):min(a),max(b){}
  int operator()() {            // callable like fn
    return min + rand()%(max-min+1);
  }
};
RandomInt roll(1,6);
cout << roll();  // uses operator()
```

unique_ptr — exclusive ownership

```
auto p1 = make_unique<Date>(2024,1,1);
// auto p2 = p1;          // ERROR — no copy
auto p2 = move(p1);       // transfer; p1=nullptr
// p2 destroyed → Date deleted automatically
```

shared_ptr — shared ownership (ref count)

```
auto p1 = make_shared<Date>(2024,1,1); // rc=1
shared_ptr<Date> p2 = p1;              // rc=2
p1.reset();  // rc=1; object still alive
p2.reset();  // rc=0; Date deleted
```

unique_ptr: zero overhead, move-only. shared_ptr: ref count, copyable.

Move Semantics

```
string a = "Hello";
string b = move(a);  // b owns data; a is empty
v.push_back(move(s));// no copy — O(1) transfer
```

After move: source is valid but unspecified (don't read it).
***Rule of Five:** if you define destructor → also define copy ctor, copy assign, move ctor, move assign.*

STRUCTURAL SIGNATURES (for design problems)

Pattern	Key structural signature
Template Method	Abstract base has non-virtual template method that calls pure virtual steps
Strategy	Context holds Strategy* field; set_strategy() method; delegates to strategy->do_it()
Observer	Subject has vector<Observer*>; attach/detach/notify; Observer has update()
State	Context holds State*; action methods do: state = state->action(). State methods return next State*
Iterator	Abstract: has_next()+next(). Concrete per collection type. Client uses only abstract interface
Visitor	Node::accept(Visitor&) calls v.visit_NodeType(this). Visitor has one visit_X() per node type
Adapter	Inherits expected interface; holds reference to incompatible object; translates method calls
Facade	One class, one or few public methods; creates + coordinates all subsystem objects internally
Composite	Component base; Leaf (no children); Composite has vector<Component*> + add(). Both inherit same base
Decorator	Inherits AND holds Component*. Overrides with: return my_val + wrapped->method()
Factory Method	Creator base has pure virtual create_X(). Subclass overrides to return concrete product
Abstract Factory	Factory base has multiple make_A(), make_B(). Concrete factory returns consistent family
Singleton	Private constructor. Static get_instance() returns static local instance. Delete copy/assign

ALL 13 PATTERNS — CATEGORY SUMMARY

Pattern	Cat	Core idea (one line)
Template Method	Beh	Fixed skeleton, variable steps via inheritance
Strategy	Beh	Swappable algorithm via composition
Observer	Beh	1→many notification on state change
State	Beh	Delegate behavior to current state object
Iterator	Beh	has_next/next over any collection
Visitor	Beh	New ops without modifying hierarchy
Adapter	Str	Interface conversion for incompatible types
Facade	Str	Simple front door to complex subsystem
Composite	Str	Leaf+group tree, uniform interface
Decorator	Str	Wrap to add behavior dynamically
Factory Method	Cre	Subclass decides what to instantiate
Abstract Factory	Cre	Create a consistent product family
Singleton	Cre	One instance, global access point

THREADING SUMMARY

Need	Use
Protect critical section	`lock_guard<mutex>` (RAII)
Multiple concurrent readers	`shared_lock<shared_mutex>`
Exclusive write	`unique_lock<shared_mutex>`
Thread-safe primitive	`atomic<T>`
Sleep until condition met	`condition_variable`
Voluntary CPU release	`this_thread::yield()`

***Deadlock prevention:** Always acquire mutexes in the same global order across all threads.*
***lock_guard vs manual:** lock_guard auto-releases even on exception. Manual unlock can be skipped.*

LAB INDEX

Lab	Concept
15.1–15.8	Recursion, Backtracking
16.1–16.4 + DiningPhil	Threading (race, mutex, RW, PC, deadlock)
2.1–2.5	SRP (Books)
4.1–4.4	OCP (Vehicles)
5.7–5.9	Law of Demeter
8.1–8.2	Template Method (Reports)
8.3–8.4	Strategy (Sports)
9.2	Factory Method
9.3	Abstract Factory
10.2	Adapter (Fans)
10.5	Facade (Funds)
11.2	Iterator (Players)
11.4	Visitor (Results)
12.2	Observer (Stats)
13.2	State (TicketMachine)
14.1	Singleton (ExecPass)
14.4	Composite (CostReport)
14.6	Decorator (Enhanced)
Lambda-1–3	Lambdas + STL
Functor-*	Functors
Smart*Pointer	unique_ptr, shared_ptr
MoveSemantics-*	Move semantics